

The following text is a SparkNotes-style summary of the learnings I acquired over my time working as an embedded engineer. It is not an exhaustive list but would serve as a good starting point in the event I return to the embedded domain.

Time

Time in an embedded system is broken into two distinct categories: monotonic time and wall clock time. Monotonic time is always increasing and usually tied to the reference clock of the system. This clock is useful for measuring timeouts and durations. In contrast, wall time provides the user-facing time of day. This is useful when computing events tied to times of day or rough synchronization between systems not on the same hardware clock.

Principle of Least Privilege

Every user, program, or process should only have the minimum level of access (permissions, rights, or privileges) necessary to perform its tasks—and nothing more. This reduces the attack surface of the system. For example, if a piece of malware is run on a low-privilege account it can't immediately take over the system. The principle of least privilege also limits the blast radius of unintentional mistakes in our software.

(Analogy from AI) Imagine you hire a plumber to fix your sink.

- With **least privilege**, you give them access only to your kitchen.
- Without it, you hand them keys to your entire house, safe, and car—unnecessary and risky.

User Space vs Kernel Space

The distinction between user space and kernel space is the modern operating systems implementation of Least Privilege. Execution of software in kernel space comes with higher privilege which allows the software to access hardware directly, manage memory, schedule processes, handle I/O, and so on; however, comes with the risk that if the software faults it will take the OS down with it. In contrast, user space programs have lower privilege which minimizes the blast radius of any problems, but can therefore not access many of the resources from kernel space. It is a common design pattern to write lightweight kernel space drivers which interface with the hardware in a deterministic and performant fashion and expose only the necessary interfaces to user space.

Kernel Driver

Developing a kernel driver is similar to any other piece of software but comes with some nuance. At a high level, kernel code is usually developed as a module to allow for faster iteration before it is ultimately built into the kernel as a driver. Kernel drivers can take arguments like any other software application; to do so, one defines a module in the device tree and adds the relevant values there. It is very common for a kernel driver to access memory mapped registers to control certain logic blocks in the silicon itself. To associate the driver with the correct block of memory the starting address of the memory mapped block is declared in the device tree along with the other variables. Effort should be given to minimize the number of variables the customer of the driver needs to know as this makes the driver less portable. It is also of the highest importance that the resources are shared properly, make correct use of mutexes, semaphores, atomic variables, and other related data structures.

Some key tools to facilitate the development of these drivers for linux are:

- lsmod: list modules
- modprobe: load modules
- insmod: insert modules
- rmmod: remove modules
- dmesg: see logs from modules
- yocto: build infrastructure used to produce linux images

Inter-Process Communication

Although not strictly related to the embedded domain, inter-process communication (IPC) is the mechanism by which various user space applications can share data. There are a large variety of these mechanisms but they can range in quality from the cursed (i.e. reading the journalctl logs of a service) to the more robust mechanisms such as a unix domain socket.

Troubleshooting

Well my friend I hope you remember how to use an oscilloscope, or at the very least a logic analyser. An industry standard at the time of writing this is the Saleae Logic Pro 16. All-in-all it's pretty straight forward, connect the probes to the trace you want to measure (if you don't know the trace, track down the schematic or the Electrical Engineer responsible for the pcb), and hit run. The oscilloscope can do a whole bunch of fancy things but you will most likely only need to play with the trigger functionality to capture the signal in question. In the event the problem is in the software domain I would recommend using GDB to get further insight into the problem.

Design Tenets

When working in the embedded space the tenets I would design around are performance, latency, and simplicity. Performance is key when running multiple pieces of software since the cpu has finite bandwidth and will ultimately starve processes if a higher priority task refuses to release context. Latency is critical to embedded systems, especially real-time embedded systems, as decisions rely on inputs and when these inputs are delayed the result has lower accuracy or has become irrelevant. These tenets should be weighed - at user discretion - against simplicity. Simple software is easier to debug, maintain, and build upon. Spaghetti code which is optimized for a certain environment may be great for a demo but will fundamentally impair the team in the long run.

Project Management

Although, again, not unique to embedded engineering there are a few notes about project management I would like to highlight. Proper project management is based on a select few core principles. These principles are as follows.

- Clear Milestone(s)
- Deliverables for Milestone(s)
- Owners for Deliverables
- Regular Status Meetings

The misuse of these principles can have severe consequences. At its best milestones are delayed or missed due to poor alignment across stakeholders. At its worst there is no ownership over deliverables and deliverables themselves are elusive and unclear due to a mismanagement or miscommunication of milestones. These deliverables are dropped only to be rediscovered at critical phases in product development and immediately prioritized to the detriment of all involved. This leads to lower job satisfaction, higher attrition, and lower team morale; it does not have to be this way.